Controlled Token-Efficient Speculation for Large Language Model Agents

Anonymous ACL submission

Abstract

Speculative decoding has been adapted to language model agents to accelerate long-horizon task completion. Agent-level speculation can reduce latency, but expanding multiple speculative branches incurs substantial token overhead. To address this token overhead, we introduce the *Controlled Token-Efficient Speculation* framework, integrating multi-step speculative trees with lightweight MLP controllers under lossless verified reuse. In this framework, pre-draft gating adaptively controls draft budget prior to generation, whereas post-draft selection prunes low-confidence branches before downstream execution. Furthermore, our theoretical analysis models the mechanics of latency savings via path coverage and execution overlap, while quantifying token savings from early gating and selection. Across E-commerce service, Chess, and HotpotQA, our controllers reduce speculative overhead while preserving acceleration, decreasing child-expansion tokens by 27.9% in service APIs, lowering the token ratio from 4.80× to 3.39× in Chess, and achieving a 12.6% wall-time reduction on HotpotQA with 31.0% token savings over fixed-width speculation.

1 Introduction

Large language model (LLM) agents increasingly execute long-horizon tasks through repeated cycles of reasoning, tool use, and environment interaction (Yao et al., 2023; Schick et al., 2023). These agentic actions are executed sequentially, as subsequent decisions depend on environmental feedback from earlier actions (e.g., retrieving search evidence before determining the next query). Over long trajectories, these sequential dependencies accumulate substantial latency, while repeated model calls incur substantial inference costs.

Speculative execution is a common strategy in computer systems for reducing the waiting time imposed by sequential dependencies. Processors predict future execution paths and execute instructions before branch conditions are resolved, discarding speculative results when predictions are incorrect (Lam and Wilson, 1992). Similarly, speculative decoding uses a lightweight draft model to propose tokens for parallel verification by a larger target model, reducing inference latency (Leviathan et al., 2023; Chen et al., 2023). It has evolved from verifying a single token chain to evaluating tree-structured token branches, increasing the chance of accepting draft continuations within a single forward pass (Miao et al., 2024; Chen et al., 2024). However, extending this paradigm to agents is challenging: agent actions involve tool execution and environmental observations, and subsequent decisions depend on feedback that token-level verification alone cannot provide. Recent studies attempt to overlap latency by speculatively generating downstream actions and plans while the target model or action is still executing (Hua et al., 2025; Guan et al., 2026; Ye et al., 2026).

To maintain high acceptance rates amid complex decision spaces, agent-level speculative execution can retain multiple candidate branches. However, prior work has mainly focused on wall-clock speedup, leaving the substantial token-budget overheads induced by aggressive speculation less explored. In Chess, our reproduced Speculative Actions baseline (Ye et al., 2026) with a width of three actions consumes 4.8 times the token budget of non-speculative execution in real-time end-to-end online evaluations, because a substantial portion of tokens is spent on speculative actions that are never reused. To address this issue, we propose to retain valuable multi-step continuations under a strictly controlled speculative token budget.

Our intuition is that allocating computation to speculative branches is only worthwhile when those actions have a high probability of being accepted by the target model. To this end, we design lightweight MLP controllers that assist the specula-

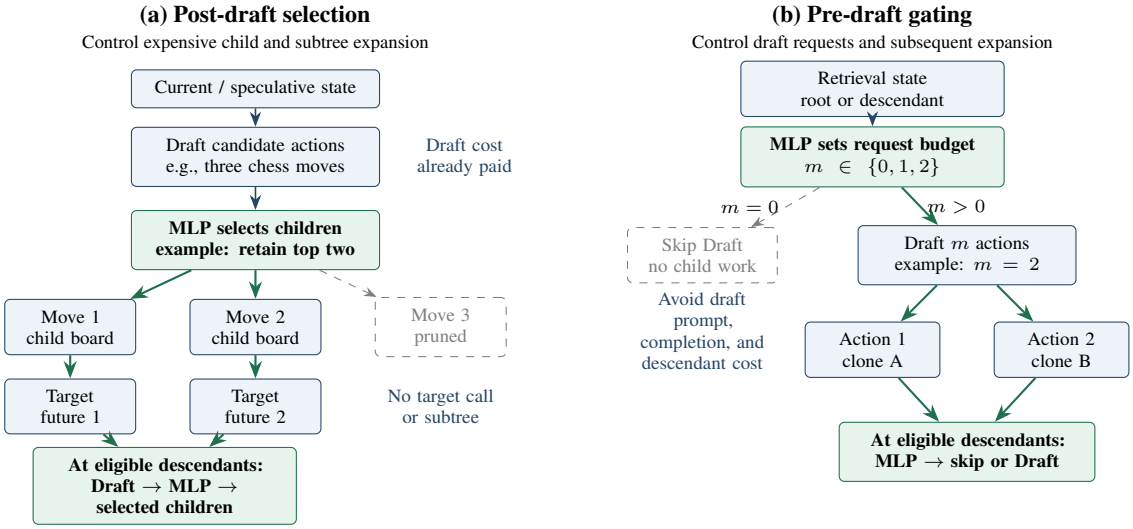


Figure 1: **Two placements of learned speculative control.** **(a)** Post-draft selection filters candidates before costly child expansion. **(b)** Pre-draft gating sets the drafting budget, avoiding draft and descendant costs when $m = 0$. Green boxes denote MLP decisions; dashed gray arrows indicate work not launched. Control repeats independently at eligible descendants. The authoritative actor runs in parallel: an exact action match in a compatible state promotes the matching child and retains valid descendants; otherwise, normal execution proceeds without waiting for unrelated speculation.

tive drafting trees by state and candidate features of the agent state to estimate where speculative computation is most likely to be useful. The MLPs regulate drafting before or after candidate generation: they can control whether and how much to draft, and selectively retain promising candidates for downstream target-model computation. Controlled trees concentrate speculative resources on higher-value continuations, balancing branch coverage against token cost. Once verified by the target model, the tree re-roots to the accepted action and continues along the retained branch, carrying the execution head start into subsequent decisions.

Specifically, we design the *Controlled Token-Efficient Speculation* framework, which integrates lightweight MLP controllers into speculative trees for adaptive **post-draft selection** and **pre-draft gating**. As illustrated in Figure 1, both post-draft selection and pre-draft gating operate to minimize speculative token overhead at distinct cost boundaries: while post-draft selection prunes low-confidence candidates after candidate generation to avoid expensive subtree expansions, pre-draft gating intervenes prior to drafting to eliminate generation expenditures when speculative gains are low. Together with exact-match validation, they ensure token-efficient acceleration without compromising execution exactness.

Our theoretical analysis relates path coverage to multi-step reuse opportunities and shows acceleration depends on useful execution overlap exceeding added overhead. It quantifies how post-draft selection avoids downstream expansion costs, while pre-draft gating saves candidate-generation costs.

We evaluate post-draft selection on E-commerce and Chess to limit downstream expansion, and pre-draft gating on HotpotQA to avoid unnecessary draft calls, tailoring our control strategy to each setting's cost structure. For service APIs, selecting two of four candidates increases accepted API depth from 3.33 to 3.58 over fixed width three and reduces child-expansion tokens by 27.9%. In complete 80-step Chess runs, low-budget control lowers the token ratio from 4.80× to 3.39× with similar time savings. On 50 HotpotQA questions, pre-draft gating reduces mean wall time by 12.6%, using 21.4% and 31.0% fewer tokens than fixed-width policies with one and two candidates, respectively.

Our contributions are threefold:

- **Method.** We propose a speculative-tree framework with lightweight MLP controllers for post-draft selection and pre-draft gating, retaining useful multi-step continuations while controlling speculative token expenditure under verified reuse.
- **Theoretical analysis.** We relate path coverage and execution overlap to latency savings and quantify the drafting and expansion costs avoided by the two controllers.

- **Empirical evaluation.** Across E-commerce, Chess, and HotpotQA, we show that our framework improves speculative tree acceptance and reduces speculative token overhead while improving online acceleration for LLM agents.

2 Related Work

Speculative decoding. Token-level speculative decoding preserves the target distribution through verification (Leviathan et al., 2023; Chen et al., 2023). SpecInfer verifies token trees in parallel (Miao et al., 2024), Sequoia optimizes tree structure (Chen et al., 2024), and EAGLE-2 uses context-dependent confidence to build dynamic draft trees (Li et al., 2024). Agent trees differ because edges invoke heterogeneous tools or entire model requests, with stateful observations, variable latency, and external concurrency limits. Their useful work cannot generally be verified in one batched target-model pass.

Agent speculation and planning. Interactive Speculative Planning considers efficient planning with user interaction (Hua et al., 2025). Dynamic Speculative Planning learns a latency–cost trade-off through online reinforcement learning (Guan et al., 2026). Speculative Actions (Ye et al., 2026) introduces action-level verification. We build on that execution principle, studying lightweight learned control at candidate-generation and downstream-expansion boundaries within persistent trees. We compare against fixed policies in the evaluated implementations.

Tool-using agents. ReAct interleaves reasoning and environment interaction (Yao et al., 2023), while Toolformer learns when and how to invoke external APIs (Schick et al., 2023). We study scheduling around this loop in HotpotQA (Yang et al., 2018), and around service API calls in retail tasks derived from τ-bench (Yao et al., 2025). The controller does not replace the actor's task policy. Candidate and path coverage measure reuse opportunities; answer quality and task success remain separate evaluation targets.

3 Controlled Token-Efficient Speculation

We present Controlled Token-Efficient Speculation, a framework accelerating sequential agent decision-making while maintaining strict token efficiency. The draft model proposes candidate continuations, the controllers regulate generation and child admission, and the runtime schedules speculative execution. Authoritative target-model actions determine which compatible speculative results are reusable, enabling high execution overlap without wasteful token overhead.

3.1 Speculative Trees: Draft, Control, and Verify

Tree construction. Symbolically, let $s_t = (H_t, e_t)$ denote the committed state, where H_t is the interaction history and e_t is the environment state. At step t, the target model samples an action $a_t \sim \pi_T(\cdot \mid s_t)$. Executing a_t yields an observation o_t and updates the environment state to e_{t+1}. The next committed state is $s_{t+1} = (H_{t+1}, e_{t+1})$, where $H_{t+1} = H_t \circ (a_t, o_t)$. Each non-root action node $v = (s_v, \hat{a}_v)$ stores a candidate action $\hat{a}_v$ and its preceding branch state $s_v = (H_v, e_v)$. Executing $\hat{a}_v$ produces a successor state s_v^+, from which child actions are proposed.

With a root storing the committed state s_t, speculative execution asynchronously expands candidate trajectories during target model verification or action latency. An MLP controller adaptively regulates draft generation and candidate filtering. Each retained candidate creates an action node with an isolated environment snapshot, allowing speculative execution and downstream computation without altering the committed state. This process recursively applies to eligible descendants, yielding a multi-step speculative tree. The tree expansion is constrained by runtime parameters: breadth B, depth D and total action-node count N. Breadth captures alternative action hypotheses at a decision point and depth enables temporal execution overlap ahead of authoritative verification.

Execution procedure. Figure 2 shows the dynamic execution loop of the framework. Starting from the committed root state, the system executes target model decision-making in parallel with controller-regulated speculative tree expansion, and the MLP controller selectively admits draft requests or candidate children. When the target model returns an action, the runtime checks the root's children for an exact canonical action match, including arguments, and compatible state dependencies. A valid match permits reuse of completed or in-flight execution, provided that the resulting observation and transition are valid for the committed state.

After transition validation, the matching child's

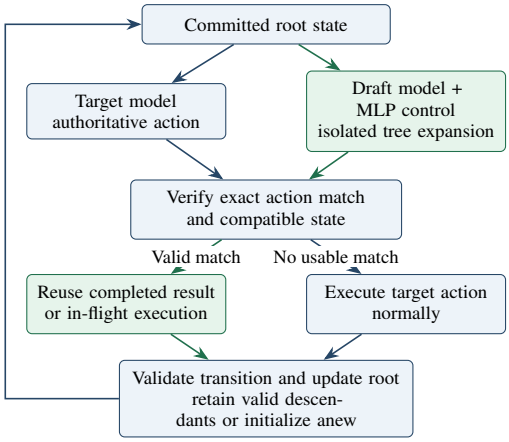


Figure 2: **Controlled token-efficient tree speculation.** Target-model execution and controlled speculative expansion run in parallel. Verified matches reuse prepared work; otherwise, normal execution proceeds. Root updates preserve valid descendants for subsequent decisions.

successor state becomes the new committed root context. Compatible completed or in-flight descendants are preserved to retain their computational head start, while unreachable siblings and invalid descendants are discarded. If no child qualifies for reuse, the target action executes normally without waiting for unrelated speculation, and its resulting state initializes the new root, allowing target-model decision and controlled speculative expansion to repeat. After migration, relative depths are recomputed from the new root, while compatible completed or in-flight work retains its progress.

Lossless reuse. It is worth emphasizing that despite relying on multi-branch speculative expansion and dynamic tree pruning, our framework operates as a lossless inference scheme. Controllers allocate speculative computation without changing authoritative target-model decisions or verification. Speculation runs in isolated snapshots without affecting the committed trajectory. With unchanged target sampling and valid, distribution-equivalent reused observations and transitions, each committed transition preserves the non-speculative conditional distribution. For reuse conditions and the transition-kernel argument, please refer to Appendix A.1.

3.2 Post-Draft Selection and Pre-Draft Gating

Both controllers operate recursively at eligible expansion points, using the committed state at the root or the successor state s_v^+ at an action node v. They intervene before different costs are incurred and allocate speculative work within the runtime limits. The controllers learn from recorded trajectories: service scoring uses candidate-match supervision, Chess uses utility-oracle supervision, and HotpotQA balances candidate coverage against drafting costs. Task-specific training details appear in Appendices B, C, and D.

Post-draft selection. At a node v, the draft model first generates a candidate set $\mathcal{A}(v)$. An MLP then uses state and candidate features to retain a subset $\mathcal{B}_\theta(v) \subseteq \mathcal{A}(v)$ before child execution and expansion begin. The controller can score and rerank individual candidates, as in service APIs, or choose how many draft-ranked candidates to retain, as in Chess. Only retained children receive downstream computation, and the procedure repeats at eligible descendants. This placement avoids rejected child computations and the subtrees they would generate.

Pre-draft gating. Before invoking the draft model at node v, an MLP selects a request budget $m_\theta(v) \in \{0, \ldots, B\}$ using only pre-request information, such as state features, historical hits, and timing estimates. If $m_\theta(v) = 0$, the runtime skips the draft request and launches no children from that node. Otherwise, the draft model is requested to produce $m_\theta(v)$ distinct candidates, whose child work is admitted within the runtime limits. The gate is evaluated again at eligible descendants. Used in HotpotQA, this placement can avoid both candidate-generation costs and subsequent subtree expansion.

4 Latency and Token-Cost Analysis

For our proposed framework, we theoretically address two key questions: how much latency speculative trees can eliminate through execution overlap, and how the controllers bound tree growth to avoid token costs. To this end, we relate path coverage to reusable opportunities, derive latency savings from speculative head starts, and characterize the costs avoided by each controller placement.

4.1 Execution Overlap

Path coverage. Let $L \in \{0, \ldots, D\}$ denote the length of consecutive authoritative action transitions covered by a retained speculative tree up to maximum action depth D. Defining q_j as the conditional coverage probability at action layer j given that all preceding layers are covered, the expected

speculative horizon $\mathbb{E}[L]$ is given by:

$$\mathbb{E}[L] = \sum_{d=1}^{D} \prod_{j=1}^{d} q_j. \tag{1}$$

This formulation requires no independence assumption across action layers. $\mathbb{E}[L]$ characterizes the theoretical coverage over retained action hypotheses. In practice, actual reuse requires valid observation returns and compatible state dependencies. Additionally, acceleration requires reusable operations to start early to yield execution overlap.

Local overlap. To quantify the latency reduction yielded by speculative execution, we model the time overlap between speculative operations and target verification. Let A, G, and W denote target-model, draft-model, and tool-execution durations. Non-speculative execution takes $T_{\text{seq}} = A + W$. If the target and draft models launch concurrently, a matching speculative tool call starts at G and finishes at $G + W$. Because authoritative execution requires both target confirmation and tool completion, a usable early match ($G < A$, indicated by I_{hit}) finishes at $T_{\text{hit}} = \max(A, G+W)$. Assuming equivalent tool durations, no resource contention, and normal execution on a miss, the resulting time savings are:

$$S = I_{\text{hit}} \min\{W, (A - G)_+\}, \tag{2}$$

where $(x)_+ = \max(x, 0)$. Speculation thus overlaps at most W units of tool execution with the target model's remaining computation. More generally, any reusable descendant work with head start h saves $\min(W, h)$, even if unfinished at verification.

End-to-end latency. To characterize global net acceleration, we formulate the expected episode-level latency $\mathbb{E}[T_\theta]$ over sequential authoritative segments indexed by t:

$$\mathbb{E}[T_\theta] = \mathbb{E}[T_0] - \mathbb{E}\left[\sum_t I_t \min(W_t, h_t)\right] + \mathbb{E}[O], \tag{3}$$

where T_0 is the non-speculative baseline time, and O captures critical-path overheads from control, verification, and resource contention. For each segment t, I_t denotes verified reuse, $h_t \geq 0$ its head start, and W_t the reusable duration. Accounting solely for critical-path savings without double-counting, net acceleration requires that reusable overlap exceeds $\mathbb{E}[O]$, proving that greater coverage or depth expansion alone is insufficient.

4.2 Token Efficiency

To formally quantify how the controllers optimize resource consumption, we analyze how managing token cost directly bounds the system overhead in our end-to-end latency framework. Without constraints, unconstrained speculative trees suffer from combinatorial explosion, scaling exponentially with breadth b and depth D. To balance exploration coverage against speculative token cost, our approach deploys two kinds of controls.

Post-draft savings. For candidate set $\mathcal{A}(v)$ proposed from committed states or successor s_v^+, selection retains a subset $\mathcal{B}_\theta(v) \subseteq \mathcal{A}(v)$. Letting $C_{\text{sub}}(v, a)$ denote the downstream execution cost of child action a, post-draft selection yields token savings of:

$$\Delta C_{\text{post}}(v) = \sum_{a \in \mathcal{A}(v) \setminus \mathcal{B}_\theta(v)} C_{\text{sub}}(v, a). \tag{4}$$

This captures avoided child computations and descendant requests from pruned candidates, while leaving generation costs invariant. The comparison assumes a fixed candidate set and plan; realized episode-level savings further reflect adaptive scheduling and controller overhead.

Pre-draft savings. To curb upstream expenditure, pre-draft gating intercepts requests before generation. At expansion point v, let P_v denote the input-token cost, $O_v(m)$ the output-token cost for m candidates, and $g_v = \mathbf{1}\{m_\theta(v) > 0\}$ the gating indicator. Expected draft usage is computed as:

$$\mathbb{E}[C_D(v)] = \mathbb{E}\big[g_v\big(P_v + O_v(m_\theta(v))\big)\big]. \tag{5}$$

Relative to a fixed positive budget b, the expected token reduction at the same node is:

$$\mathbb{E}[\Delta C_D(v)] = \mathbb{E}[P_v + O_v(b) - g_v\big(P_v + O_v(m_\theta(v))\big)]. \tag{6}$$

A zero budget skips the request entirely, eliminating draft-request costs and preventing new subtrees while preserving valid existing speculative work. While a smaller positive budget limits branching breadth, zero-budget skipping completely avoids

Draft model	Policy	Precision	Recall	F1	Calls	Hits	Tokens
Gemini-3.5-Flash	Keep all	19.02	37.23	25.18	184	35	998,354
	Fixed top-3	19.86	29.79	23.83	141	28	706,515
	`e-commerce-mlp`	**38.24**	27.66	**32.10**	68	26	**324,352**
GPT-5.5	Keep all	27.19	32.98	29.81	114	31	527,639
	Fixed top-3	27.66	27.66	27.66	94	26	429,828
	`e-commerce-mlp`	**37.50**	25.53	**30.38**	**64**	24	**293,947**

Table 1: Static E-commerce service evaluation on held-out τ-bench retail tasks.

Policy	API acc.	Exact-call acc.	Accepted depth	Replay tokens
Fixed rank 1	74.19	58.06	2.67	60,560
Fixed rank 2	80.65	61.29	2.83	132,194
Fixed rank 3	87.10	62.90	3.33	241,911
`e-commerce-mlp`	**88.71**	62.90	**3.58**	**168,884**
Keep all (4)	90.32	64.52	3.92	377,188

Table 2: Dynamic evaluation accompanying E-commerce service: authoritative-path replay across τ^3-bench domains with GPT-5.5 draft-model candidates.

P_v, making it effective in long-context regimes where input tokens dominate draft expenses.

Post-draft selection avoids downstream expansion costs, while pre-draft gating additionally avoids candidate-generation costs. Both placements regulate speculative computation within the runtime limits. Further execution details and supplementary analysis are provided in Appendices A.2 and A.3.

5 Experiments

We evaluate our controlled speculative execution framework across three benchmarks: E-commerce service, Chess, and HotpotQA retrieval. For E-commerce and Chess where draft candidates exhibit relatively high hit probabilities but subtree expansion is costly, we deploy post-draft selection to retain high-potential candidates while suppressing downstream overhead. For HotpotQA, where candidate generation itself is expensive due to long input contexts, we employ pre-draft gating to skip low-confidence draft requests, avoiding both upstream generation costs and subsequent execution. We detail these two control configurations below.

5.1 Post-Draft Selection: E-Commerce and Chess

Tasks and controllers. For E-commerce service, we evaluate our framework by combining static exact-call selection on held-out τ-bench (Yao et al., 2025) retail tasks with dynamic read-only path replay across τ^3-bench domains (Barres et al., 2026; Shi et al., 2026), using Gemini-3.5-Flash and GPT-5.5 as backbone models. The `e-commerce-mlp` controller with two ReLU hidden layers scores candidates using rank, API identity, argument structure, and confidence or agreement features, retaining the top two out of four under the main dynamic policy. We compare learned selection with keep-all and fixed top-three retention in static evaluation, and fixed-rank retention of one, two, or three candidates and keep-all retention in dynamic replay. Replay measures retained path coverage and prompt usage, while isolated child execution measures actual expansion tokens. Exact-call matching requires identical API names and JSON arguments, upon which trajectory execution advances. More details appear in Appendix B.

Chess evaluation includes predefined fixed-FEN positions for short-horizon testing and complete 80-step runs for sustained expansion and repeated root migration. We use Gemini-3.1-Pro as the target model and GPT-5.4-mini as the draft model. The `chess-mlp` controller ($18 \to 64 \to 32 \to 4$) selects $m \in \{0, 1, 2, 3\}$ draft-ranked children using candidate confidence and execution-state features, retaining valid speculative work after verified root migration. We compare low- and high-budget variants against width-three Speculative Actions (SA-3) (Ye et al., 2026) and a depth-three chain, measuring latency reduction and token usage relative to non-speculative execution. Further settings and evaluation details appear in Appendix C.

E-commerce service results and analysis. Static evaluation (Table 1) shows that learned selection improves exact-call F1 over fixed top-three

Policy	Time saved (%)	Token ratio	Hit rate (%)
SA-3	8.27	4.79×	30.0
`chess-mlp` low	**13.65**	**3.78**×	30.0
`chess-mlp` high	13.64	5.41×	**35.0**
Chain 1×3	3.01	5.18×	10.0

Table 3: Fixed-FEN Chess evaluation with 20 cells per policy and three decisions per cell. Time savings and token ratios are relative to non-speculative execution.

Policy	Time saved (%)	Token ratio	Hits
SA-3	16.02	4.80×	80/240
Chain 1×3	13.51	4.14×	79/320
`chess-mlp` low	15.92	**3.39**×	68/240
`chess-mlp` high	**17.49**	4.59×	84/240

Table 4: Complete 80-step online Chess runs. Hits report matched decisions over evaluated steps.

from 23.83% to 32.10% with Gemini candidates and from 27.66% to 30.38% with GPT candidates, while reducing prompt tokens by 54.1% and 31.6%, respectively. Higher precision with fewer retained calls suggests that the controller concentrates expansion on more reliable candidates, and this consistent pattern across separately trained controllers supports the usefulness of candidate scoring beyond draft rank.

Dynamic replay (Table 2) extends this advantage to multi-step coverage: retaining two reranked candidates increases accepted API depth from 3.33 to 3.58 relative to fixed rank three, with unchanged exact-call accuracy and 30.2% fewer replay tokens. A replication with a different target–draft model pair shows the same pattern (Appendix B). These results indicate better candidate selection can compensate for narrower branching, preserving useful paths while avoiding unused subtree expansion.

Chess results and analysis. Fixed-FEN evaluation (Table 3) shows that low-budget `chess-mlp` reduces the token ratio from 4.79× to 3.78× while increasing recorded time savings from 8.27% to 13.65%, with unchanged hit rate. The high-budget variant obtains more hits but similar time savings at greater cost, indicating that hit frequency alone does not determine acceleration: useful branches must also start early enough to offset their computation cost.

Complete runs confirm that the cost advantage persists through repeated expansion and root migration (Table 4). Low-budget selection reduces the token ratio from 4.80× to 3.39× while maintaining approximately the same time savings as SA-3.

Policy	Precision	Recall	Set-F1	Tokens/state
Fixed-1	48.48	48.48	48.48	2,109.6
Fixed-2	24.75	49.49	32.99	2,131.1
`hotpotQA-mlp`	**53.03**	44.41	48.34	**1,681.9**

Table 5: Static HotpotQA evaluation. Precision, recall, and set-F1 are matching percentages against recorded target-model actions.

The high-budget variant reaches 17.49% savings at 4.59× tokens. The fixed chain is less efficient than low-budget selection in both evaluations, indicating that depth alone is insufficient. More details appear in Appendix C.

5.2 Pre-Draft Gating: HotpotQA Retrieval

Task and controller. HotpotQA (Yang et al., 2018) evaluates multi-hop retrieval through static candidate evaluation and online runs on 50 questions. We use `gpt-5.4-medium` as the target model and `gpt-5-mini` as the draft model, with `Search`, `Lookup`, and `Finish` actions. The `hotpotQA-mlp` controller uses pre-request history, timing, and hit features to select $m \in \{0, 1, 2\}$ candidates at root and eligible descendant nodes, skipping drafting when $m = 0$. Speculative retrieval runs in isolated Wiki clones, with reuse requiring exact action matching and compatible state dependencies. Baselines include non-speculative execution (Pure), Speculative Actions (Ye et al., 2026) Fixed-1, and Fixed-2; all speculative policies share the persistent-tree runtime and verification rules. Static evaluation measures candidate-set quality and draft-token usage; online evaluation measures latency, total tokens, and answer quality. Further details appear in Appendix D.

Results and analysis. Table 5 shows that `hotpotQA-mlp` retains nearly the same set-F1 as Fixed-1 (48.34% versus 48.48%) with 20.3% fewer draft tokens. Compared with Fixed-2, it reduces draft tokens by 21.1% and improves set-F1 from 32.99% to 48.34%. The gate skips 18.43% of draft requests, avoiding their complete input and output costs. In contrast, adding a second candidate under fixed drafting increases recall by only 1.01 percentage points. The results indicate limited marginal coverage from wider drafting in this test and support request admission as a way to reduce cost, with some loss of recall.

On the 50 questions (Table 6), `hotpotQA-mlp` reduces mean wall time by 12.6% relative to Pure,

Policy	EM	Answer F1	Wall time (s/question) Mean	Median	P90	Tokens/question	Time saved (%)
Pure	**0.620**	**0.742**	33.76	23.77	62.31	**7,994**	–
Fixed-1	0.520	0.654	30.75	24.40	55.70	16,677	8.9
Fixed-2	0.540	0.685	**28.93**	21.68	**48.67**	19,015	**14.3**
`hotpotQA-mlp`	**0.620**	0.741	29.52	**21.40**	56.78	**13,116**	12.6

Table 6: Online HotpotQA evaluation on 50 questions.

from 33.76 to 29.52 seconds, with matched observed EM of 0.620 and nearly identical answer F1. It uses 21.4% fewer total tokens than Fixed-1 while also lowering mean latency, and 31.0% fewer than Fixed-2 with only 0.59 seconds greater mean latency.

This trade-off is consistent with the two effects of pre-draft control. Skipping requests removes long-context input costs as described in Equation 5, while admitted branches still overlap useful work with target-model computation as in Equation 2. Persistent descendants retain this head start across root migrations and the observed answer-quality agreement with Pure supports the online result.

Summary. Across the three workflows, the Controlled Token-Efficient Speculation framework allocates computation before unused work becomes expensive. Post-draft selection limits subtree expansion while preserving useful coverage and online acceleration; pre-draft gating additionally avoids drafting costs. Both placements reduce speculative token overhead while improving coverage or preserving useful acceleration, supporting allocation based on expected reuse and execution overlap.

6 Ablation Study

E-commerce ablation study. On the same GPT-5.5 dataset as the main experiment, feature ablations at retained width two show that adding API, argument, and domain information increases mean accepted depth from 2.833 to 3.417 and two-step coverage from 66.67% to 78.33%. In the cross-model replication, holding the candidate scorer fixed, dynamic retention preserves the same mean accepted depth and one-/two-step coverage as retaining three scored candidates while reducing replay prompt tokens by 24.5%. These results support both informed candidate ranking and selective branch retention as effective components of post-draft control. Complete results appear in Appendix E.1.

Chess ablation study. Feature-group masking supports the controller's use of both candidate reliability and execution-state information: masking confidence-related features reduces selected utility by 23.3%, while masking latency and resource features reduces it by 83.0%. The fixed-FEN comparisons in Table 3 further show that selective branching improves recorded time savings by 10.64 percentage points over the depth-three chain while lowering the token ratio from 5.18× to 3.78×. Low-budget control also matches the high-budget variant's average time savings with a 30.1% lower token ratio. These results support allocating speculative computation according to candidate usefulness and execution conditions. Additional results and reuse diagnostics appear in Appendix E.2.

HotpotQA Retrieval. On a separate 50-question online evaluation, we compare the full gate with variants disabling request skipping (No-skip), retaining only request admission (Skip-only), or randomizing budgets (Random-budget). The full gate uses 13.9% and 13.5% fewer total tokens than No-skip and Random-budget, respectively, and skip-only achieves similar token cost. These results support request skipping as an important source of token savings and show lower resource usage than the tested random policy. Complete results appear in Appendix E.3.

7 Conclusion

We presented Controlled Token-Efficient Speculation, combining persistent speculative trees with lightweight MLP control. Post-draft selection limits its downstream expansion, while pre-draft gating avoids unnecessary draft requests. Verified root migration preserves completed and in-flight work under lossless reuse conditions. Our analysis relates path coverage and execution overlap to latency savings and characterizes avoidable token costs. Across E-commerce, Chess, and HotpotQA, learned control reduces speculative overhead while preserving useful coverage and acceleration.

Limitations

Our evaluation covers three complementary agent workflows with selected model pairs and speculative budgets; broader evaluations across tasks, models, and execution horizons would further characterize the observed benefits. Hosted API response times may also vary with network conditions, service load, and provider updates. Although policy execution order is balanced in the fixed-FEN Chess and online HotpotQA evaluations, latency gains reflect the measured operating conditions, with token usage and request counts providing complementary measures of resource consumption.

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik R Narasimhan. 2026. τ^2-bench: Evaluating conversational agents in a dual-control environment. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 6736–6782. PMLR.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *Preprint*, arXiv:2302.01318.

Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable and robust speculative decoding. In *Advances in Neural Information Processing Systems*, volume 37, pages 129531–129563. Curran Associates, Inc.

Yilin Guan, Qingfeng Lan, Fei Sun, Dujian Ding, Devang Acharya, Chi Wang, William Yang Wang, and Wenyue Hua. 2026. Dynamic speculative agent planning. In *The Fourteenth International Conference on Learning Representations*.

Wenyue Hua, Mengting Wan, Jagannath Vadrevu, Ryan Nadel, Yongfeng Zhang, and Chi Wang. 2025. Interactive speculative planning: Enhance agent efficiency through co-design of system and user interface. In *International Conference on Learning Representations*, volume 2025, pages 14256–14283.

Monica S. Lam and Robert P. Wilson. 1992. Limits of control flow on parallelism. In *Proceedings of the 19th Annual International Symposium on Computer Architecture*, ISCA '92, page 46–57, New York, NY, USA. Association for Computing Machinery.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7421–7432, Miami, Florida, USA. Association for Computational Linguistics.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 932–949, New York, NY, USA. Association for Computing Machinery.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc.

Quan Shi, Alexandra Zytek, Pedram Razavi, Karthik R Narasimhan, and Victor Barres. 2026. τ-knowledge: Evaluating conversational agents over unstructured knowledge. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 112443–112471. PMLR.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A benchmark for Tool-Agent-User interaction in real-world domains. In *International Conference on Learning Representations*, volume 2025, pages 9965–10017.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. 2026. Speculative actions: A lossless framework for faster AI agents. In *The Fourteenth International Conference on Learning Representations*.

A Supplementary Tree Analysis

A.1 Conditions for Verified Reuse

Canonicalization normalizes representation, such as JSON key order, without merging different argument values or ordered lists. Reuse also requires compatible state dependencies: read-only execution alone does not guarantee that a cached observation is current. A speculative model request can replace authoritative computation only when its effective history and generation configuration agree with the authoritative request. Selecting among sampled outputs by content or completion time must not bias the target-model distribution.

Let $F(s, a, o)$ denote the state update defined in Section 3.1, U a measurable set of successor states, and $K(do \mid s, a)$ the normal observation kernel. If the reused or fallback observation has this same kernel conditional on the route selected by verification, the committed transition kernel is

$$Q(U \mid s) = \sum_a \pi_T(a \mid s) \cdot \int \mathbf{1}\{F(s, a, o) \in U\} K(do \mid s, a). \tag{7}$$

Both runtimes therefore induce the same trajectory distribution by induction from a common initial state. This argument assumes isolated speculation and unchanged target-model sampling.

After root migration, dependency checks determine which descendants remain valid, and relative depths are reset before further expansion. Unreachable work is canceled where possible. Cancellation does not recover tokens already consumed, and in-flight requests may continue to occupy resources.

A.2 Coverage and Timing Derivations

The tail-sum identity $L = \sum_{d=1}^{D} \mathbf{1}\{L \geq d\}$ and the probability chain rule give Equation 1. With constant conditional coverage $q_j = q$,

$$\mathbb{E}[L] = \begin{cases} q(1 - q^D)/(1 - q), & 0 \leq q < 1, \\ D, & q = 1. \end{cases} \tag{8}$$

Early misses reduce all subsequent coverage terms. These probabilities describe retained paths; actual reuse additionally requires valid observations and state dependencies.

For a usable early match, Equation 2 follows from

$$A + W - \max(A, G + W) = \min(W, A - G), \quad G < A \tag{9}$$

Its expectation must retain dependence between matching events and durations; multiplying a hit rate by a saving calculated from average durations is generally insufficient. If normal and speculative durations differ, the saving becomes

$$S_{\text{hit}} = A + W_0 - \max(A, G + W_s), \tag{10}$$

which can be negative under contention.

For descendant work launched at ℓ_t and needed at τ_t, the remaining wait is $[W_t - h_t]_+$, where $h_t = [\tau_t - \ell_t]_+$. Root migration preserves this progress. Equation 3 counts each reusable computation once, using head starts on the evolving authoritative schedule. Concurrent savings must be evaluated on the critical path rather than summed independently.

A.3 Draft and Request Accounting

For the same node distribution and a fixed positive budget b, the draft-cost reduction can be separated into skipped requests and narrower admitted requests:

$$\mathbb{E}[\Delta C_D] = \mathbb{E}[(1 - g_v)(P_v + O_v(b))] + \mathbb{E}[g_v(O_v(b) - O_v(m_\theta(v)))]. \tag{11}$$

This assumes equal input length across positive widths. If width instructions change that length, replace P_v by the corresponding width-dependent input cost. Whole-run comparisons must also account for different visited nodes.

Actual model usage is counted once per issued request:

$$C_\theta = \sum_{r \in \mathcal{R}_\theta} (I_r + O_r). \tag{12}$$

Charged reasoning tokens are included according to provider fields without double counting. Unused completed requests and charged partial requests contribute to usage; cached reuse adds no second charge.

B Supplementary E-commerce Service Details

B.1 Data Construction and Supervision

Static evaluation. Both draft-model datasets use a task-level split of 86/13/16 training/validation/test retail tasks. The Gemini-3.5-Flash dataset contains 2,153/426/478 training/validation/test candidate nodes. The 297 non-test rollouts contain 832 positive and 1,747 negative examples; the test set contains 48 completed rollouts and 327 turns. The

Policy	Accepted depth	D1 (%)	D2 (%)	Replay tokens
Fixed rank 1	2.000	66.7	60.0	70,700
Fixed rank 2	2.067	66.7	60.0	141,400
Fixed rank 3	2.533	80.0	73.3	256,983
Keep all	3.000	93.3	93.3	430,616
`e-commerce-mlp`	2.747	92.0	85.3	206,228

Table 7: Cross-model authoritative-path replay. Learned-policy results are averaged over five initializations; replay tokens measure prompt usage.

GPT-5.5 dataset contains 1,252/179/240 candidate nodes, with 560 exact-read positives across all 1,671 nodes. The two controllers are trained separately without mixing their model-specific datasets.

A positive static label requires a read-only candidate's canonical tool name and sorted JSON arguments to match a call in the next recorded target-model turn. The controller uses 64- and 32-unit ReLU hidden layers and a scalar reliability output. Features include candidate rank, API identity, argument structure, confidence, entropy, margin, generation-cost metadata, and cross-rollout agreement where available. Each candidate is treated as a binary supervised example. Static retention thresholds are selected on the validation set and frozen for held-out evaluation.

Dynamic evaluation. The GPT-5.5 dynamic dataset spans airline, banking knowledge, and retail domains (Barres et al., 2026; Shi et al., 2026), with task-disjoint splits of 41/7/15 tasks and 164/28/62 recorded states. Each state has four read-only candidate calls. Accuracy is measured on the 62 held-out states; accepted depth and D1/D2 use 12 eligible trajectories.

Dynamic labels indicate whether the candidate API belongs to the next authoritative API set. Features include rank, reciprocal rank, agreement signals, API identity, fixed argument-text hashes, and domain indicators. The main policy reranks the four candidates and retains two. Both static and dynamic MLPs are trained with weighted binary cross-entropy to balance the contributions of positive and negative examples, with checkpoints selected by the lowest validation mean squared error. Accepted depth measures consecutive API coverage. D1/D2 measure coverage of at least one/two consecutive transitions.

B.2 Expansion Accounting and Replication

Actual child expansion. Selected calls execute in isolated read-only snapshots. Their observations are appended to branch histories before the draft model generates next-layer proposals. Fixed rank three completes 141/141 expansions using 260,081 input and 81,848 output tokens (341,929 total). Learned width two completes 98/98 expansions using 187,587 input and 58,941 output tokens (246,528 total), a 27.9% reduction. These provider-reported totals measure expansion cost, separately from replay prompt usage.

Cross-model replication. The replication uses Gemini-3.5-Flash target-model trajectories and GPT-5.4-mini-2026-03-17 draft-model candidates, with task-disjoint splits of 41/7/15 tasks. A validation-frozen threshold adaptively admits up to three candidates per state. Table 7 reports results averaged over five controller initializations.

Compared with fixed rank three, learned selection increases mean accepted depth by 0.213 and reduces replay tokens by 19.8%.

C Supplementary Chess Details

C.1 Training and Budget Variants

Training data are collected from 40 breadth-three, depth-three runs, yielding 8,960 node records. Splitting by run gives 6,155/1,377/1,428 training/validation/test examples, preventing adjacent nodes from the same run from appearing in different splits. Records include board positions, draft candidates, submission and completion times, path acceptance, pruning outcomes, token usage, and resource occupancy.

The controller uses an $18 \to 64 \to 32 \to 4$ architecture. Inputs summarize depth, candidate rank, path probability, legal-move and candidate counts, confidence, entropy, margin, estimated target/draft latency, active nodes and futures, remaining node budget, node and worker limits, and player identity. The four outputs correspond to retaining $m \in \{0, 1, 2, 3\}$ draft-ranked children.

For each recorded node, we compute a utility

oracle for each retention action $m \in \{0, 1, 2, 3\}$:

$$U(m) = S(m) - \lambda_{\text{tok}} C_{\text{tok}}(m) - \lambda_{\text{waste}} C_{\text{waste}}(m) - \lambda_{\text{conc}} C_{\text{conc}}(m). \quad (13)$$

Here, $S(m)$ denotes estimated waiting-time savings, while the cost terms account for token usage, unused work completing after pruning, and concurrency or congestion. These action utilities provide supervision for learning how many draft-ranked children to expand, aligning branch admission with useful execution overlap and resource cost.

C.2 Evaluation and Accounting

Fixed-FEN evaluation uses ten predefined opening positions, two repetitions, and three decisions per cell. Four policies are evaluated in each of 20 comparison blocks, giving 80 completed cells. Cyclic Latin shifts balance policy execution order. No router/resource errors, tracebacks, or format retries are recorded.

Complete 80-step results aggregate three runs for SA-3 and each MLP variant, and four runs for the chain. The chain follows the highest-ranked draft move to depth three without an MLP or adaptive/deadline admission gate. SA-3 token ratios use prediction-deduplicated accounting. Combined full-length tokens are 533,178 for low-budget control, 718,739 for high-budget control, and 863,849 for the chain. These raw totals cover different numbers of runs and are not equal-work comparisons.

A fixed-FEN event-loop diagnostic records an additional 0.372 ± 0.078 seconds of savings per sample from second-child prefetching relative to top-one prefetching. Selected children are submitted 1.91 seconds early on average, illustrating how in-flight work can retain useful progress after root migration.

D Supplementary HotpotQA Details

D.1 Static Data and Controller

The static gate dataset contains 1,259/310/689 training/validation/test states. Its 11 pre-request features summarize the current step, prompt and question lengths, historical target/draft latency, estimated head start, previous prediction hits, and the preceding action type. All features are available before the current draft request.

The gate balances candidate-hit benefits against the costs of initiating speculation. Its utility is summarized as

$$U(s, m) = B_{\text{hit}}(s, m) - \lambda_{\text{draft}} C_{\text{draft}}(s, m) - \lambda_{\text{child}} C_{\text{child}}(s, m), \quad (14)$$

where B_{hit} represents candidate-hit or reuse benefits, C_{draft} accounts for draft input and output tokens, and C_{child} accounts for speculative clone execution. The evaluated gate selects $m \in \{0, 1, 2\}$ using only pre-request features. Choosing $m = 0$ avoids the entire draft request and its downstream child work.

On the 689-state static test set, the draft-request rate is 81.57%, corresponding to 18.43% skipped requests, and mean requested width is 0.84. Static candidate-set metrics measure coverage of recorded target-model actions rather than complete-task answer quality.

D.2 Online Configuration

The completed 50-question evaluation uses `gpt-5.4-medium` as the target model and `gpt-5-mini` as the draft model, both with temperature 0.1 and top-p 0.9. Execution allows at most six authoritative steps and uses tree limits $B = 2$, $D = 2$, and $N = 6$ non-root action nodes. The controller selects $m \in \{0, 1, 2\}$ at eligible expansion points. Policy order rotates by question.

Speculative actions execute in isolated Wiki clones. A clone is promoted only after an exact canonical action match and compatible state-dependency checks; valid descendants retain their completed or in-flight work after root migration. If no usable match is available, authoritative execution proceeds without waiting for unrelated draft or sibling work.

D.3 Request and Expansion Breakdown

Table 8 reports model-request usage, while Figure 3 decomposes speculative child execution and reports descendant draft calls.

Learned gating concentrates the reduction in unused child computation. Compared with Fixed-2, it reduces executed children from 6.72 to 2.36 and wasted children from 5.58 to 1.38 per question, while promoted children decrease from 1.14 to 0.98. Relative to Fixed-1, it retains a similar number of promoted children (0.98 versus 1.12) with fewer wasted children (1.38 versus 2.16). Draft calls also decrease from 5.22 under Fixed-2 to 2.74 under learned gating. The gate retains 1.02 descendant draft calls per question, compared with 1.60 and

Policy	Target calls	Draft calls	Target tokens	Draft tokens	Median total tokens
Pure	3.38	0.00	7,994	0	6,814
Fixed-1	3.40	4.04	7,962	8,715	13,984
Fixed-2	3.26	5.22	7,491	11,524	16,706
`hotpotQA-mlp`	3.24	2.74	7,529	5,587	12,344

Table 8: Request-level breakdown for the completed 50-question evaluation. Calls and target/draft tokens are per-question means; the final column reports median total tokens per question.

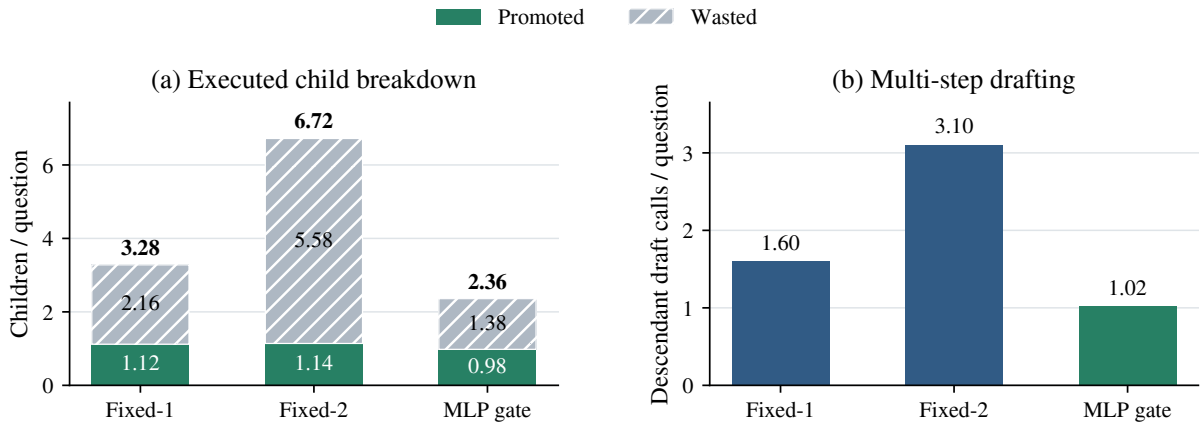


Figure 3: HotpotQA speculative-work breakdown on the completed 50-question evaluation. (a) Executed children are decomposed into promoted and wasted children. Wasted children are executed speculative children that are not promoted to the authoritative trajectory. (b) Draft calls at descendant nodes. All values are per-question means. MLP gate denotes `hotpotQA-mlp`.

3.10 for Fixed-1 and Fixed-2, confirming that selective admission preserves multi-step expansion while reducing unnecessary work.

D.4 Paired Wall-Time Comparisons

The learned gate is faster than Pure on 36/50 questions. Its mean paired wall-time difference is -4.24 seconds and its median paired difference is -2.58 seconds. Learned gating uses fewer tokens on 38/50 questions relative to Fixed-1 and 40/50 relative to Fixed-2.

E Ablation Study

E.1 E-commerce

Evaluation setup. We examine candidate features and retained width on the same task-disjoint GPT-5.5 dataset used in Table 2, comprising 41/7/15 training/validation/test tasks and 164/28/62 recorded states. Feature variants share the candidate pool, network architecture, training procedure, and retained width. Width variants reuse each trained scorer and change only the number of retained candidates. Learned-policy results are averaged over five initializations. API and exact-call coverage are evaluated on all 62 held-out states; accepted depth and D2 use 12 eligible trajectories. Accepted depth measures consecutive API coverage, and D2 denotes coverage of at least two consecutive transitions. Unlike the single-initialization main result in Table 2, the learned results below are averaged over five controller initializations.

Contribution of candidate information. Table 9 compares progressively richer candidate representations at retained width two. Rank-only scoring reproduces the original draft ordering. Adding API identity increases mean accepted depth from 2.833 to 3.333 and API coverage from 80.65% to 85.48%. Argument and domain information further increase depth to 3.417 and two-step coverage to 78.33%. Overall, the main feature configuration improves API coverage by 6.45 percentage points and D2 by 11.67 percentage points over rank-only scoring. Since all variants retain two candidates, these improvements demonstrate that candidate information helps allocate the same branching capacity to more reliable continuations. Additional generation-cost features yield a similar operating point, supporting the compact representation used in the main experiment.

Features	API acc. (%)	Exact acc. (%)	Accepted depth	D2 (%)	Replay tokens
Rank only	80.65	61.29	2.833	66.67	132,194
Rank + API	85.48	62.58	3.333	75.00	161,274
Rank + API + arguments	86.77	62.58	3.367	76.67	162,796
Rank + API + arguments + domain	**87.10**	62.90	**3.417**	**78.33**	164,318
Above + generation cost	86.13	**63.23**	3.383	76.67	162,796

Table 9: Candidate-feature ablation on the GPT-5.5 service dataset at retained width two. Learned results are five-initialization means.

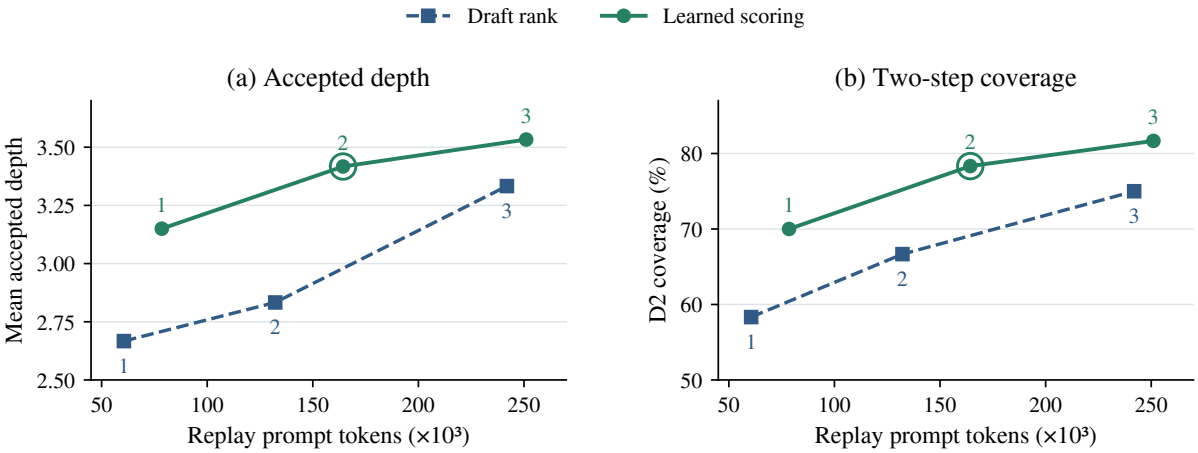


Figure 4: Coverage–cost trade-offs across retained widths on the GPT-5.5 service dataset. Points are labeled by retained width, with learned width two highlighted by circles; lines connect the tested configurations. Learned scoring uses the main feature configuration and reports five-initialization means. Replay tokens measure prompt usage.

Retained-width trade-off. Figure 4 compares replay prompt usage with accepted depth and two-step coverage at retained widths one, two, and three. Learned scoring improves both path-coverage measures at every tested width. API coverage also increases from 74.19%, 80.65%, and 87.10% under draft ranking to 80.97%, 87.10%, and 88.39%, respectively. Exact-call coverage increases from 61.29% to 62.90% at width two and from 62.90% to 64.19% at width three; at width one, it is 56.77% versus 58.06% under draft ranking. Learned width two retains 96.7% of the width-three mean accepted depth with 34.5% fewer replay tokens. Compared with draft rank three, it matches API and exact-call coverage while achieving greater accepted depth and D2 with 32.1% fewer replay tokens, supporting width two as an effective coverage–cost trade-off.

Selective retention with a fixed scorer. We further isolate the retention rule using the cross-model replication in Table 7, with Gemini-3.5-Flash target trajectories and GPT-5.4-mini draft candidates. For each initialization, we hold the full candidate scorer fixed and compare retaining its top one, two, or three candidates with a validation-selected dynamic threshold. Dynamic retention always retains the highest-scored candidate and conditionally admits the second and third candidates; the threshold is selected on validation to approach a predefined mean width of 2.5.

Table 10 shows that dynamic retention achieves the same mean accepted depth, D1, and D2 as fixed width three, while reducing replay tokens from 273,032 to 206,228, a 24.5% reduction. Compared with fixed width two, dynamic retention increases depth from 2.533 to 2.747 and D2 from 80.00% to 85.33%. This comparison isolates the contribution of selective admission: the same candidate scores support broader path coverage where additional branches are useful while avoiding uniform three-child expansion.

E.2 Chess

Candidate reliability and execution-state features. We examine two feature groups central to the controller's allocation decisions: candidate reliability, comprising confidence, entropy, and margin, and execution state, comprising latency esti-

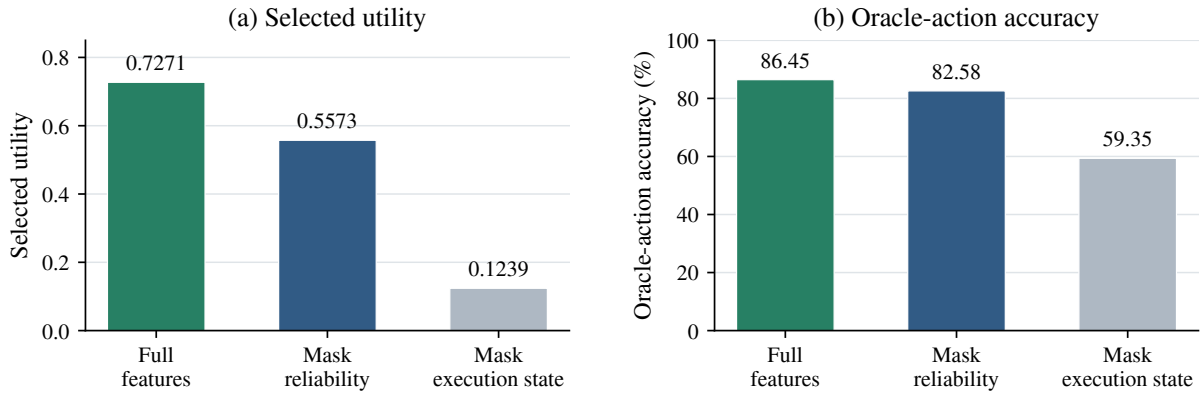


Figure 5: Feature-group masking on 155 held-out chess nodes using a fixed low-budget controller. Each masked group is replaced by its training mean. Panels report (a) selected utility and (b) oracle-action accuracy.

Retention rule	Depth	D1 (%)	D2 (%)	Replay tokens
MLP fixed-1	2.453	84.00	77.33	83,836
MLP fixed-2	2.533	86.67	80.00	170,220
MLP fixed-3	2.747	92.00	85.33	273,032
MLP dynamic	2.747	92.00	85.33	206,228

Table 10: Retention-rule ablation in the cross-model replication. Each initialization uses the same scorer across retention rules. Results are five-initialization means; D1/D2 measure coverage of at least one/two consecutive transitions.

Policy	Time saved	Ratio	Hit	After-prune
Depth-three chain	3.01	5.18×	10.0	49.1
`chess-mlp low`	**13.65**	**3.78**×	30.0	**29.9**
`chess-mlp high`	13.64	5.41×	**35.0**	46.4

Table 11: Fixed-FEN policy and budget diagnostics. Time savings, hit rates, and after-prune shares are percentages; after-prune shares use combined tokens as the denominator.

mates and resource information. Evaluation uses the same 155 held-out node records and a fixed trained low-budget controller. Each tested group is replaced by its training mean, corresponding to zero after standardization. Selected utility measures the stored utility of the controller's chosen action, combining expected waiting-time savings with token, wasted-work, and concurrency costs. Oracle regret is the difference between the maximum available utility and the selected utility, while oracle-action accuracy measures agreement with the utility-maximizing action.

Figure 5 shows complementary contributions from candidate reliability and execution-state information. Masking reliability reduces selected utility by 23.3% and increases oracle regret from 0.0279 to 0.1977, demonstrating the value of distinguishing promising continuations. Masking execution state reduces utility by 83.0%, increases regret to 0.6311, and lowers oracle-action accuracy from 86.45% to 59.35%, highlighting the importance of available overlap and resource capacity. Together, these results support allocating speculative computation using both candidate quality and execution conditions.

Selective branching and budget efficiency. We supplement the fixed-FEN results in Table 3 with after-prune token usage, defined as tokens from requests that complete after their branches become invalid. All three policies use the same ten initial positions, two repetitions per position, and three decisions per evaluation cell. Table 11 connects the observed time and token trade-offs to this component of unused computation.

Relative to the fixed chain, low-budget selection increases recorded time savings by 10.64 percentage points, reduces the token ratio by 27.0%, and lowers the after-prune share from 49.1% to 29.9%. This comparison supports selective branching over fixed top-one lookahead. The low- and high-budget controllers provide complementary learned operating points: low-budget control achieves nearly identical average time savings with a 30.1% lower token ratio and a 16.5-percentage-point lower after-prune share. These findings connect the controller's budget advantage to reduced unused branch computation and reinforce the importance of useful execution overlap rather than hit frequency alone.

Execution overlap and persistent reuse. Event-loop diagnostics further support retaining useful computation across root migration. Across

60 FENs with five repetitions each, second-child prefetching provides an average incremental waiting-time saving of 0.372 seconds per sample relative to top-one prefetching; the 18 held-out FENs show a corresponding gain of 0.374 seconds. In a ten-FEN smoke evaluation, five second-candidate hits involve child requests submitted an average of 1.91 seconds before the root returns. These requests remain in flight at verification and retain their computational head start after migration, demonstrating that partially completed work can already reduce subsequent waiting.

An audit of 77 online trajectories records 1,376 selected children promoted to the next root, including 84 whose requests were still in flight at migration. Completed requests provide immediately reusable results, while ongoing requests continue from their existing progress, so subsequent execution waits only for their remaining computation rather than restarting the request. Root migration therefore preserves the head start obtained by overlapping future computation with the preceding target decision. A further 20 matched cases occur at termination or the execution cutoff. These observations confirm that persistent reuse carries both completed results and partial execution progress across verified transitions, supporting the overlap mechanism described in Section 3.1.

E.3 HotpotQA Retrieval

Setting and variants. We evaluate four policies on a separate 50-question online batch, reporting completed causal trajectories rather than offline candidate replay. Full denotes the complete pre-draft gate. No-skip disables zero-budget decisions while retaining positive-budget selection. Skip-only retains request admission but removes adaptive positive-width selection. Random-budget replaces learned state-dependent budget selection with randomized budgets. These variants examine request skipping, positive-width adaptation, and learned budget allocation, respectively.

Complete results. Table 12 reports token usage and drafting activity; Table 13 reports latency and answer quality. Full uses 12,052 total tokens per question, compared with 14,003 for No-skip and 13,933 for Random-budget, corresponding to reductions of 13.9% and 13.5%. Draft tokens decrease by 31.2% and 28.4%, respectively, while draft calls decrease from 4.06 and 4.04 to 3.08. Skip-only uses slightly fewer total tokens than Full, with comparable observed answer quality.

Policy	Total tokens	Draft tokens	Calls	Width
Full	12,052	4,244	3.08	0.930
No-skip	14,003	6,173	4.06	1.082
Skip-only	11,698	3,831	2.70	0.758
Random-budget	13,933	5,925	4.04	1.143

Table 12: HotpotQA gating ablations on 50 questions per policy. All entries are means; token and request counts are per question. Width denotes the reported mean budget width.

Policy	Mean wall	EM	Answer F1
Full	27.246	0.480	0.641
No-skip	27.184	0.460	0.621
Skip-only	30.239	0.480	0.653
Random-budget	28.043	0.500	0.647

Table 13: Latency and answer quality for the same ablation batch. Wall times are seconds per question.

Contribution of request skipping. Disabling zero-budget decisions increases both draft calls and draft tokens without a demonstrated latency benefit. The increase in draft tokens accounts for approximately 98.9% of the total-token increase relative to Full. This pattern is consistent with the cost mechanism in Equation 5: declining a draft request avoids its input and output costs, rather than merely shortening its candidate list. The ablation therefore supports request admission as an important component of token-efficient speculation.

Positive-width adaptation and randomized budgets. Full and Skip-only operate at similar token costs, but Full achieves a lower mean wall time, from 30.24 to 27.25 seconds, with the same observed EM of 0.480. This suggests that allowing larger positive budgets at selected states can preserve useful execution overlap beyond request admission alone. Compared with Random-budget, Full reduces total tokens by 13.5% and draft tokens by 28.4%, while maintaining a slightly lower mean latency. Its lower draft-call count and mean width indicate more selective resource use under the learned policy.

Interpretation. The ablations support request skipping as an important source of token savings: Full substantially reduces drafting costs relative to No-skip while maintaining nearly identical mean latency. Relative to Skip-only, it achieves lower observed mean latency at a similar token cost; relative to Random-budget, it uses fewer tokens and re-

quests. Together, these results support the intended trade-off of pre-draft control: avoiding unnecessary generation while allocating enough speculative work to retain useful overlap with authoritative execution.

F Artifact Licensing

Our experiments build on existing benchmarks and evaluation environments, including τ-bench, chess, and HotpotQA. We respect the original licenses and usage restrictions of these resources. The planned release focuses on our own implementation and does not redistribute third-party datasets, model weights, or benchmark artifacts. Any third-party dependencies remain subject to their respective licenses and terms of use.